

Centro Universitário Senac-RS

ADS - Análise e Desenvolvimento de Sistemas / SPI - Sistemas para Internet

Unidade Curricular: Qualidade de Software

Prof.: Luciano Zanuz

Atividade PBL – Aula 9

Testes Unitários Automatizados e TDD – LocalEats

Contexto

Após planejar, especificar e executar testes de forma estruturada, a equipe de Qualidade do sistema **LocalEats** precisa evoluir sua abordagem de QA:

👉 Sair de testes manuais e documentais para **testes automatizados no código**

Até agora:

- Os testes foram planejados e descritos
- A execução foi manual ou simulada

Agora, o desafio é:

👉 **Garantir qualidade diretamente no código, de forma automatizada**

O sistema ainda apresenta problemas como:

- Regras de negócio inconsistentes
- Falhas após alterações no código (regressões)
- Dificuldade em validar rapidamente mudanças
- Código difícil de manter e evoluir

A equipe precisa garantir que:

“As regras de negócio sejam validadas automaticamente e continuamente.”

Para isso, será adotado:

- Testes unitários automatizados
- Prática de TDD (Test-Driven Development)

👉 Link para o sistema LocalEats: <https://local-eats-unisenac.vercel.app/>

Objetivo da Atividade

Aplicar, de forma prática e orientada ao desenvolvimento:

- Escrita de testes unitários automatizados
- Validação de regras de negócio
- Aplicação do ciclo TDD (Red → Green → Refactor)
- Refatoração orientada por testes



Importante

- É necessário implementar código
- Foco em **funções e regras de negócio (backend ou lógica pura)**
- Não testar interface gráfica (frontend)
- Utilizar a linguagem/stack do projeto do grupo
- Caso o sistema não esteja implementado, criar funções simuladas
- O mais importante é o **raciocínio e aplicação do TDD**



Tarefas

Elaborem um documento contendo:

♦ 1. Funcionalidade escolhida

Selecionar **uma regra de negócio do sistema por integrante do grupo**.



Cada integrante deverá trabalhar com **uma funcionalidade diferente**.



Para esta atividade, utilizem uma das funcionalidades sugeridas abaixo (ou equivalente, desde que siga o mesmo nível de clareza).



Funcionalidades sugeridas



1. Cálculo do total do pedido com valor mínimo



O que faz

- Soma os valores dos itens do pedido
- Verifica se o pedido atinge o valor mínimo



Problema que resolve

Evita pedidos inválidos que não atendem às regras do restaurante



Importância

Regra central do fluxo de compra

Regras de negócio

- Total = soma dos itens
- Se total < valor mínimo → erro
- Caso contrário → pedido válido

Exemplo (Python)

```
def calcular_total_pedido(itens, valor_minimo):  
    total = sum(item["preco"] for item in itens)  
  
    if total < valor_minimo:  
        raise ValueError("Valor mínimo do pedido não atingido")  
  
    return total
```

2. Aplicação de desconto percentual

O que faz

Aplica um desconto sobre o valor total do pedido

Problema que resolve

Permite promoções e campanhas

Importância

Impacta diretamente o preço final

Regras de negócio

- Desconto deve estar entre 0% e 100%
- Valor final não pode ser negativo

3. Cálculo de taxa de entrega

O que faz

Calcula o valor da entrega com base na distância

Problema que resolve

Padroniza cobrança de entrega

Importância

Impacta custo final do pedido

 Regras de negócio

- Distância até 3 km → taxa fixa
- Acima disso → taxa proporcional
- Distância negativa → erro

 4. Validação de login

 O que faz

Valida credenciais do usuário

 Problema que resolve

Garante segurança de acesso

 Importância

Protege dados e funcionalidades do sistema

 Regras de negócio

- Usuário e senha devem coincidir
- Campos não podem estar vazios

 5. Cálculo de tempo estimado de entrega

 O que faz

Calcula o tempo estimado com base na distância

 Problema que resolve

Melhora a previsibilidade para o usuário

 Importância

Impacta experiência do cliente

 Regras de negócio

- Tempo base + tempo por km
- Distância inválida → erro

 2. Testes Unitários

Criar **mínimo de 3 testes por integrante**

👉 Cada teste deve conter:

- Nome descritivo
- Cenário testado
- Dados de entrada
- Resultado esperado
- Código do teste

👉 Incluir obrigatoriamente:

- Pelo menos 2 cenários de sucesso (happy path)
- Pelo menos 1 cenário de erro ou borda

💻 Exemplo (Python)

```
# Nome descritivo:
# Deve calcular corretamente o total do pedido quando valor mínimo é atingido

# Cenário testado:
# Valida se a função retorna corretamente o total do pedido
# quando a soma dos itens é maior ou igual ao valor mínimo exigido.

# Dados de entrada:
# itens = [{"preco": 10}, {"preco": 20}]
# valor_minimo = 15

# Resultado esperado:
# Retornar 30
# Não deve gerar erro

def test_deve_calcular_total_quando_valor_minimo_atingido():
    # Arrange (preparação)
    itens = [{"preco": 10}, {"preco": 20}]
    valor_minimo = 15

    # Act (execução)
    resultado = calcular_total_pedido(itens, valor_minimo)

    # Assert (validação)
    assert resultado == 30
```

◆ 3. Aplicação do TDD

Aplicar o ciclo completo em **uma funcionalidade**

Red

- Escrever o teste antes da implementação
- Demonstrar falha

Green

- Implementar o mínimo necessário

Refactor

- Melhorar o código mantendo os testes passando

👉 Apresentar código + explicação

◆ 4. Refatoração

👉 Melhorar:

- Legibilidade
- Nomes
- Organização
- Duplicações

👉 Explicar as melhorias realizadas

◆ 5. Execução dos Testes

👉 Informar:

- Total de testes
- Quantos passaram
- Quantos falharam

👉 Registrar:

- Evidência (print ou log)

◆ 6. Reflexão no contexto do LocalEats

Responder:

- Foi difícil escrever testes antes do código?
- O TDD ajudou no desenvolvimento?
- Os testes aumentaram a confiança no código?
- O que melhorariam?
- Como isso ajuda no projeto do grupo?



Entregável

Formato: arquivo Markdown (.md)

Entrega: repositório do grupo no GitHub

- /aula-09-testes-unitarios-tdd.md

Trabalho individual ou em grupo (até 4 integrantes)



Exemplo

<https://github.com/lucianozanuz/pbl-qualidade-software-2026-1/blob/main/pbl/aula-09-testes-unitarios-tdd.md>



Avaliação (Rubrica – Unisenac-RS)



D — Não atingiu as competências mínimas

- Não implementa testes corretamente
- Não aplica TDD
- Código ausente ou incorreto
- Sem execução de testes



C — Atingiu parcialmente as competências

- Testes incompletos
- TDD superficial
- Baixa clareza



B — Atingiu plenamente as competências

- Testes corretos e organizados
- TDD aplicado corretamente
- Execução consistente



A — Atingiu as competências com excelência

- Testes claros e bem estruturados
- TDD completo (Red, Green, Refactor)
- Refatoração bem justificada
- Evidências organizadas
- Reflexão crítica

👉 Diferenciais:

- Código limpo
- Boas práticas
- Pensamento profissional



Dica final

Para obter conceito A, vocês devem:

- Aplicar TDD de verdade (não inverter a ordem)
- Escrever testes claros e bem nomeados
- Validar cenários de erro
- Refatorar com justificativa
- Demonstrar entendimento e não apenas fazer funcionar

👉 Mentalidade esperada:

“Se eu mudar esse código amanhã, meus testes vão garantir que nada quebre?”